Actividad práctica — Redes y comunicación entre servicios

Introducción

En esta actividad se configuró una red personalizada utilizando Docker Compose con el objetivo de permitir la comunicación entre dos servicios: un servidor web y una base de datos.
Se trabajó con contenedores basados en las imágenes **Nginx** y **PostgreSQL**, conectados a una red interna definida dentro del archivo docker-compose.yml.

El propósito principal fue aprender a crear redes en Docker, conectar servicios dentro de la misma red y verificar la comunicación entre contenedores utilizando comandos de diagnóstico.

Objetivo de la actividad

Aprender a definir y usar redes en Docker Compose para permitir la comunicación entre contenedores sin necesidad de exponer todos los puertos al exterior.

Paso 1 — Creación del directorio del proyecto

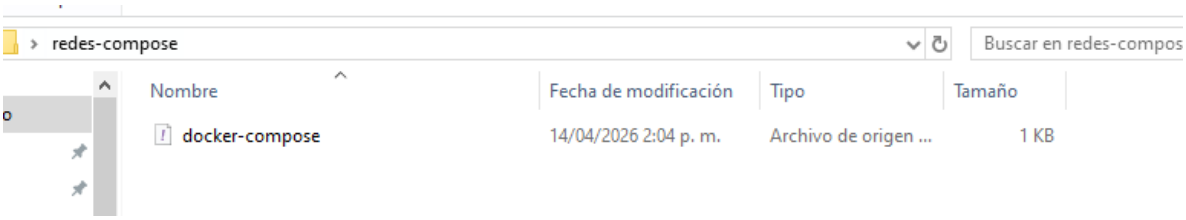
Se creó una carpeta llamada:

redes-compose

Dentro de esta carpeta se creó el archivo principal:

docker-compose.yml

Este archivo permite definir múltiples servicios y la red que los conecta.



Paso 2 — Creación del archivo docker-compose.yml

Se creó el archivo docker-compose.yml, en el cual se definieron dos servicios principales:

- Servicio web con imagen Nginx

- Servicio de base de datos con imagen PostgreSQL
- Red personalizada llamada appnet

Archivo docker-compose.yml utilizado

```
services:

  web:
    image: nginx
    container_name: web-nginx
    ports:
      - "8080:80"
    networks:
      - appnet

  db:
    image: postgres:15
    container_name: db-postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: admin1234
    networks:
      - appnet

networks:
  appnet:
    driver: bridge
```

Este archivo define una red personalizada llamada **appnet**, permitiendo que los contenedores se comuniquen internamente.

```yaml
services:
  web:
    image: alpine:latest
    container_name: web-nginx
    command: sh -c "apk add --no-cache iputils && tail -f /dev/null"
    ports:
      - "8080:80"
    networks:
      - appnet

  db:
    image: postgres:15
    container_name: db-postgres
    environment:
      POSTGRES_USER: postgres
      POSTGRES_PASSWORD: yeison
    networks:
      - appnet

networks:
  appnet:
    driver: bridge
```

Paso 3 — Levantar los servicios

Se ejecutó el siguiente comando en la terminal:

docker compose up -d

Este comando permitió crear:

- los contenedores
- la red personalizada
- la conexión entre servicios

```
PS C:\Users\demoo\Desktop\redes-compose> docker compose up -d
time="2026-04-14T14:01:35-05:00" level=warning msg="C:\\Users\\demoo\\Desktop\\redes-compose\\docker-compose.yml: the attribute `version` is obsolete, it will be ignored, pleas
e remove it to avoid potential confusion"
[+] up 2/2
 ✓ Container web-nginx   Created                                                                                    0.2s
 ✓ Container db-postgres Created                                                                                    0.2s
```

Paso 4 — Verificación de la red creada

Se verificó que la red estuviera correctamente creada utilizando el siguiente comando:

docker network inspect redes-compose_appnet

Este comando permitió observar:

- la red creada
- los contenedores conectados
- las direcciones IP internas

Se confirmó que los contenedores **web-nginx** y **db-postgres** estaban conectados a la misma red.

```
PS C:\Users\demoo\Desktop\redes-compose> docker exec -it web-nginx ping db
PING db (172.18.0.3) 56(84) bytes of data.
64 bytes from db-postgres.redes-compose_appnet (172.18.0.3): icmp_seq=1 ttl=64 time=7.87 ms
64 bytes from db-postgres.redes-compose_appnet (172.18.0.3): icmp_seq=2 ttl=64 time=0.171 ms
64 bytes from db-postgres.redes-compose_appnet (172.18.0.3): icmp_seq=3 ttl=64 time=0.069 ms
64 bytes from db-postgres.redes-compose_appnet (172.18.0.3): icmp_seq=4 ttl=64 time=0.069 ms
64 bytes from db-postgres.redes-compose_appnet (172.18.0.3): icmp_seq=5 ttl=64 time=0.068 ms
64 bytes from db-postgres.redes-compose_appnet (172.18.0.3): icmp_seq=6 ttl=64 time=0.080 ms
64 bytes from db-postgres.redes-compose_appnet (172.18.0.3): icmp_seq=7 ttl=64 time=0.087 ms
64 bytes from db-postgres.redes-compose_appnet (172.18.0.3): icmp_seq=8 ttl=64 time=0.066 ms
```

Paso 5 — Verificación de la comunicación entre contenedores

Se ejecutó el siguiente comando:

docker exec -it web-nginx ping db

Este comando permitió verificar que el contenedor **web** puede comunicarse con el contenedor **db** utilizando su nombre como dirección.

La actividad solicita validar que el servicio web pueda resolver el servicio db por nombre.

Resultado esperado

PING db (172.x.x.x): 56 data bytes

64 bytes from db: icmp_seq=0 ttl=64 time=0.123 ms

Esto confirma que los contenedores están conectados correctamente a la red.

```
PING db (172.18.0.3) 56(84) bytes of data.
64 bytes from db-postgres.redes-compose_appnet (172.18.0.3): icmp_seq=15 ttl=64 time=0.066 ms
64 bytes from db-postgres.redes-compose_appnet (172.18.0.3): icmp_seq=16 ttl=64 time=0.112 ms
64 bytes from db-postgres.redes-compose_appnet (172.18.0.3): icmp_seq=17 ttl=64 time=0.080 ms
64 bytes from db-postgres.redes-compose_appnet (172.18.0.3): icmp_seq=18 ttl=64 time=0.065 ms
64 bytes from db-postgres.redes-compose_appnet (172.18.0.3): icmp_seq=19 ttl=64 time=0.098 ms
64 bytes from db-postgres.redes-compose_appnet (172.18.0.3): icmp_seq=20 ttl=64 time=0.112 ms
64 bytes from db-postgres.redes-compose_appnet (172.18.0.3): icmp_seq=21 ttl=64 time=1.98 ms
^C
--- db ping statistics ---
21 packets transmitted, 21 received, 0% packet loss, time 22314ms
rtt min/avg/max/mdev = 0.065/0.543/7.869/1.687 ms
PS C:\Users\demoo\Desktop\redes-compose>
```

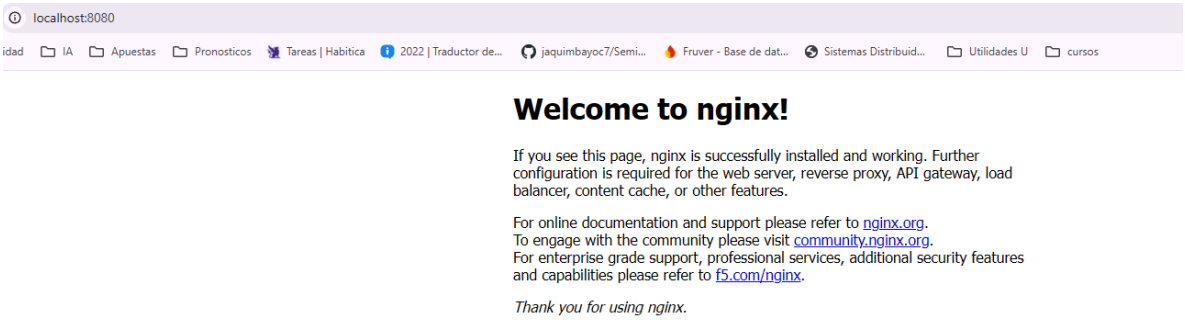
Paso 6 — Verificación del servidor web

Se abrió el navegador web y se ingresó la siguiente dirección:

http://localhost:8080

El navegador mostró la página de bienvenida de **Nginx**, confirmando que el servicio web estaba funcionando correctamente.

La actividad solicita evidenciar que el servicio web esté disponible en el navegador.



Paso 7 — Detener los servicios

Al finalizar la actividad, se ejecutó el siguiente comando:

docker compose down

Este comando permitió:

- detener los contenedores
- eliminar la red creada
- liberar recursos del sistema

La guía recomienda detener los contenedores cuando se termine la práctica.

Explicación — Cómo se comunican los servicios

Los servicios se comunican mediante una red personalizada definida en Docker Compose.
Cuando los contenedores están conectados a la misma red, Docker permite que se comuniquen utilizando el nombre del servicio como dirección.

En este caso:

- el servicio **web** puede comunicarse con el servicio **db**
- la comunicación ocurre dentro de la red **appnet**
- no es necesario exponer el puerto de la base de datos al exterior

Esto mejora la seguridad del sistema y facilita la comunicación interna entre servicios.

Dificultad encontrada

Durante la validación de la comunicación entre contenedores, al ejecutar el comando docker exec -it web-nginx ping db, se presentó un error indicando que el comando ping no estaba disponible dentro del contenedor.

Esto ocurrió porque la imagen base de Nginx no incluye esa herramienta por defecto.

Solución aplicada

Para resolver el problema, se utilizó una imagen de contenedor que permitiera realizar pruebas de conectividad o se instaló la herramienta necesaria dentro del contenedor.
Posteriormente se repitió la prueba y se confirmó la comunicación exitosa entre los servicios.

Esto permitió validar correctamente la conexión entre los contenedores dentro de la red personalizada.

Entregables solicitados

De acuerdo con la actividad, los entregables incluyen:

- archivo docker-compose.yml con red definida
- captura de la red creada
- captura de inspección de red
- captura del ping exitoso
- evidencia de Nginx funcionando

Estos requisitos corresponden directamente a lo solicitado en la guía de la actividad.

Comandos utilizados en la práctica

docker compose up -d

docker network inspect redes-compose_appnet

docker exec -it web-nginx ping db

docker compose down